FLUXO BACKEND - CÓDIGO COMPLETO
Documentação de todos os arquivos criados e alterados
Gerado em: 21 de abril de 2024

==

ARQUIVO: prisma.config.ts

```
import "dotenv/config";
import path from "path";
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  schema: path.join("prisma", "schema.prisma"),
  migrations: {
    path: path.join("prisma", "migrations"),
  },
  datasource: {
    url: env("DATABASE_URL"),
  },
});
```

==

ARQUIVO: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "moduleResolution": "node",
    "strict": false,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,
    "allowJs": true,
    "resolveJsonModule": true,
    "outDir": "dist",
    "rootDir": "."
  },
  "include": ["src/**/*.ts", "src/**/*.js"]
}
```

==

ARQUIVO: prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model User {
  id          Int       @id @default(autoincrement())
  name        String
  email       String    @unique
  password    String

  createdAt   DateTime  @default(now())
  lastLogin   DateTime?
  isActive    Boolean   @default(true)

  flows       Flow[]
  feedbacks   HotspotFeedback[]
  reputation  UserReputation?
  strategies  StrategySession[]

  @@index([email])
}

model Zone {
  id         Int               @id @default(autoincrement())
  name       String            @unique
  demand     Int               @default(0)
  flows      Flow[]
  feedbacks  HotspotFeedback[]

  @@index([name])
}

model Flow {
  id           Int        @id @default(autoincrement())
  userId       Int
  user         User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  platform     Platform
  zone         ZoneName?
  zoneId       Int?
  zoneModel    Zone?      @relation(fields: [zoneId], references: [id])
  latitude     Float?
  longitude    Float?
  placeName    String?
  placeAddress String?
  startedAt    DateTime
```

```
  finishedAt   DateTime?
  durationMin  Float?
  value        Float?
  km           Float?
  createdAt    DateTime   @default(now())

  @@index([userId])
  @@index([zoneId])
  @@index([createdAt])
  @@index([latitude, longitude])
}

model HotspotFeedback {
  id         Int          @id @default(autoincrement())
  userId     Int
  user       User         @relation(fields: [userId], references: [id], onDelete: Cascade)
  zoneId     Int
  zone       Zone         @relation(fields: [zoneId], references: [id])
  latitude   Float
  longitude  Float
  type       FeedbackType
  weight     Float
  createdAt  DateTime     @default(now())

  @@index([userId])
  @@index([zoneId])
  @@index([createdAt])
}

model UserReputation {
  id            Int      @id @default(autoincrement())
  userId        Int      @unique
  user          User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  score         Float    @default(0)
  feedbackCount Int      @default(0)
  updatedAt     DateTime @updatedAt
}

model StrategySession {
  id            Int            @id @default(autoincrement())
  userId        Int
  user          User           @relation(fields: [userId], references: [id], onDelete: Cascade)
  status        StrategyStatus
  startedAt     DateTime       @default(now())
  endedAt       DateTime?
  totalDistance Float          @default(0)
  totalTimeMin  Float          @default(0)
  totalScore    Float          @default(0)
  createdAt     DateTime       @default(now())
  updatedAt     DateTime       @updatedAt

  @@index([userId])
  @@index([status])
  @@index([createdAt])
}

enum Platform {
  ifood
  uber
  noventa_nove
  rappi
}

enum ZoneName {
  zona_sul
  zona_norte
  zona_leste
  zona_oeste
  centro
}

enum FeedbackType {
  GOOD
  BAD
  DANGER
  TRAFFIC
  HIGH
}

enum StrategyStatus {
  active
  ended
}

================================================================================

ARQUIVO: src/app.js

require("dotenv").config();

console.log("VERSION NOVA BACKEND");
console.log("Environment:", process.env.NODE_ENV || "development");

require("ts-node/register/transpile-only");
const express = require("express");
const cors = require("cors");

const prisma = require("../prisma");
```

```
const authRoutes = require("./modules/auth/routes");
const flowRoutes = require("./modules/flow/routes");
const dashboardRoutes = require("./modules/dashboard/routes");
const insightsRoutes = require("./modules/insights/routes");
const assistantRoutes = require("./modules/assistant/routes");
const copilotRoutes = require("./modules/copilot/routes");
const newRoutes = require("./routes.ts");

const app = express();

app.use(cors());
app.use(express.json());
app.use("/", newRoutes);
app.use("/", authRoutes);
app.use("/", insightsRoutes);
app.use("/", assistantRoutes);
app.use("/", copilotRoutes);
app.use("/", require("./modules/realtime/routes"));

app.get("/", (req, res) => {
  res.send("API Fluxo funcionando");
});

app.get("/health", async (req, res) => {
  try {
    await prisma.\SELECT 1;
    res.json({ status: "ok", database: "connected" });
  } catch (err) {
    res.status(500).json({ status: "error", database: "disconnected" });
  }
});

app.use("/auth", authRoutes);
app.use("/flow", flowRoutes);
app.use("/dashboard", dashboardRoutes);

const PORT = process.env.PORT || 3000;

const server = app.listen(PORT, "0.0.0.0", () => {
  console.log(Server rodando na porta \);
});

// Graceful shutdown
process.on("SIGTERM", async () => {
  console.log("SIGTERM received: closing HTTP server");
  server.close(async () => {
    console.log("HTTP server closed");
    await prisma.\();
    console.log("Prisma disconnected");
    process.exit(0);
  });
});

================================================================================

ARQUIVO: src/routes.ts

import express from "express";
import auth from "./middlewares/auth.middleware";
import geo from "./middlewares/geo.middleware";
import flowController from "./modules/flow/flow.controller";
import copilotController from "./modules/copilot/copilot.controller";
import feedbackController from "./modules/feedback/feedback.controller";
import strategyController from "./modules/strategy/strategy.controller";
import metricsController from "./modules/metrics/metrics.controller";

const router = express.Router();

router.post("/flow", auth, geo, flowController.createFlow);
router.get("/copilot", auth, geo, copilotController.getCopilot);
router.post("/feedback", auth, geo, feedbackController.submitFeedback);
router.post("/strategy/start", auth, strategyController.startStrategy);
router.post("/strategy/update", auth, strategyController.updateStrategy);
router.post("/strategy/end", auth, strategyController.endStrategy);
router.get("/metrics", auth, metricsController.getMetrics);

router.use((req, res) => {
  res.status(404).json({ success: false, error: "Endpoint não encontrado" });
});

export = router;

================================================================================

ARQUIVO: src/middlewares/auth.middleware.ts

const jwt = require("jsonwebtoken");

export default function auth(req: any, res: any, next: any) {
  const header = req.headers["authorization"];

  if (!header) {
    return res.status(401).json({ success: false, error: "Token ausente" });
  }

  const token = header.split(" ")[1];

  try {
```

```
    const decoded = jwt.verify(token, process.env.JWT_SECRET || "changeme");
    req.user = decoded;
    return next();
  } catch (err) {
    return res.status(401).json({ success: false, error: "Token inválido" });
  }
}
```

==========

ARQUIVO: src/middlewares/geo.middleware.ts

```
import { validateInsideSP } from "../services/zone.service";

export default function geo(req: any, res: any, next: any) {
  const lat = parseFloat(req.body.lat ?? req.query.lat);
  const lng = parseFloat(req.body.lng ?? req.query.lng);

  if (Number.isNaN(lat) || Number.isNaN(lng)) {
    return res.status(400).json({ success: false, error: "Latitude e longitude obrigatórias" });
  }

  if (!validateInsideSP(lat, lng)) {
    return res.status(400).json({ success: false, error: "Localização fora de São Paulo" });
  }

  req.location = { lat, lng };
  return next();
}
```

==========

ARQUIVO: src/modules/flow/flow.controller.ts

```
const flowService = require("./flow.service");

async function createFlow(req: any, res: any) {
  try {
    const userId = req.user?.id;
    if (!userId) {
      return res.status(401).json({ success: false, error: "Usuário não autenticado" });
    }

    const data = await flowService.ingestFlow(userId, {
      lat: req.body.lat,
      lng: req.body.lng,
      platform: req.body.platform,
      placeName: req.body.placeName,
      placeAddress: req.body.placeAddress
    });

    return res.json({ success: true, data });
  } catch (err: any) {
    return res.status(400).json({ success: false, error: err.message || "Falha ao salvar fluxo" });
  }
}

export default { createFlow };
```

==========

ARQUIVO: src/modules/flow/flow.service.ts

```
const prisma = require("../../../prisma");
const { getZoneFromPoint } = require("../../services/zone.service");

async function ingestFlow(userId: number, payload: any) {
  const latitude = Number(payload.lat);
  const longitude = Number(payload.lng);

  if (!Number.isFinite(latitude) || !Number.isFinite(longitude)) {
    throw new Error("Latitude e longitude inválidas");
  }

  const zone = await getZoneFromPoint(latitude, longitude);

  const flow = await prisma.flow.create({
    data: {
      userId,
      platform: payload.platform || "ifood",
      zone: zone.name,
      zoneId: zone.id,
      latitude,
      longitude,
      placeName: payload.placeName || null,
      placeAddress: payload.placeAddress || null,
      startedAt: new Date()
    }
  });

  await prisma.zone.update({
    where: { id: zone.id },
    data: { demand: { increment: 1 } }
  });

  return flow;
}
```

```
export default { ingestFlow };

================================================================================

ARQUIVO: src/modules/copilot/copilot.controller.ts

const copilotService = require("./copilot.service");

async function getCopilot(req: any, res: any) {
  try {
    const lat = Number(req.query.lat);
    const lng = Number(req.query.lng);

    if (!Number.isFinite(lat) || !Number.isFinite(lng)) {
      return res.status(400).json({ success: false, error: "Latitude e longitude obrigatórias" });
    }

    const data = await copilotService.getCopilot(lat, lng);
    return res.json({ success: true, data });
  } catch (err: any) {
    return res.status(400).json({ success: false, error: err.message || "Erro no copiloto" });
  }
}

export default { getCopilot };

================================================================================

ARQUIVO: src/modules/copilot/copilot.service.ts

const prisma = require("../../../prisma");
const { getZoneFromPoint } = require("../../services/zone.service");
const { groupNearbyPoints, calculateDensityScore, haversineDistance } = require("../../utils/cluster.util");

function normalizeIntensity(score: number) {
  return Math.max(0, Math.min(100, Math.round(score)));
}

function clusterFeedbackBoost(cluster: any, feedbacks: any[]) {
  const radius = Math.max(cluster.radius, 120);
  const nearbyFeedback = feedbacks.filter((feedback) => {
    const distance = haversineDistance(
      { lat: feedback.latitude, lng: feedback.longitude },
      cluster.center
    );
    return distance <= radius;
  });

  if (!nearbyFeedback.length) return 1;

  const totalWeight = nearbyFeedback.reduce((sum, item) => sum + item.weight, 0);
  return 1 + Math.min(0.75, totalWeight / 20);
}

async function getCopilot(lat: number, lng: number) {
  const zone = await getZoneFromPoint(lat, lng);
  const since = new Date(Date.now() - 30 * 60 * 1000);

  const flows = await prisma.flow.findMany({
    where: {
      zoneId: zone.id,
      latitude: { not: null },
      longitude: { not: null },
      createdAt: { gte: since }
    },
    orderBy: { createdAt: "desc" },
    take: 500
  });

  const feedbacks = await prisma.hotspotFeedback.findMany({
    where: {
      zoneId: zone.id,
      createdAt: { gte: since }
    }
  });

  const clusters = groupNearbyPoints(
    flows.map((flow) => ({ lat: flow.latitude, lng: flow.longitude, flowId: flow.id })),
    200
  );

  const hotspots = clusters
    .map((cluster) => {
      const score = calculateDensityScore(cluster) * clusterFeedbackBoost(cluster, feedbacks);
      return {
        lat: cluster.center.lat,
        lng: cluster.center.lng,
        intensity: normalizeIntensity(score),
        cluster
      };
    })
    .sort((a, b) => b.intensity - a.intensity)
    .slice(0, 8)
    .map(({ lat, lng, intensity }) => ({ lat, lng, intensity }));

  const zones = await prisma.zone.findMany({ orderBy: { demand: "desc" }, take: 5 });

  return {
```

```
    hotspots,
    zones: zones.map((item) => ({ id: item.id, name: item.name, demand: item.demand }))
  };
}

export default { getCopilot };

================================================================================

ARQUIVO: src/modules/feedback/feedback.controller.ts

const feedbackService = require("./feedback.service");

async function submitFeedback(req: any, res: any) {
  try {
    const userId = req.user?.id;
    if (!userId) {
      return res.status(401).json({ success: false, error: "Usuário não autenticado" });
    }

    const feedback = await feedbackService.submitFeedback(userId, {
      lat: req.body.lat,
      lng: req.body.lng,
      type: req.body.type
    });

    return res.json({ success: true, data: feedback });
  } catch (err: any) {
    return res.status(400).json({ success: false, error: err.message || "Falha ao enviar feedback" });
  }
}

export default { submitFeedback };

================================================================================

ARQUIVO: src/modules/feedback/feedback.service.ts

const prisma = require("../../../prisma");
const { getZoneFromPoint } = require("../../services/zone.service");
const reputationService = require("../../services/reputation.service");
const antifraudService = require("../../services/antifraud.service");

const BASE_WEIGHTS: Record<string, number> = {
  GOOD: 1.2,
  BAD: 0.9,
  DANGER: 1.5,
  TRAFFIC: 0.6,
  HIGH: 1.1
};

async function submitFeedback(userId: number, payload: any) {
  const allowed = await antifraudService.checkFeedbackAllowed(userId);
  if (!allowed.allowed) {
    throw new Error(allowed.reason);
  }

  const zone = await getZoneFromPoint(Number(payload.lat), Number(payload.lng));
  const reputation = await reputationService.getOrCreateReputation(userId);
  const baseWeight = BASE_WEIGHTS[payload.type] ?? 0.7;
  const finalWeight = Math.max(0.1, baseWeight * reputationService.reputationWeight(reputation) * allowed.weightFactor);

  const feedback = await prisma.hotspotFeedback.create({
    data: {
      userId,
      zoneId: zone.id,
      latitude: Number(payload.lat),
      longitude: Number(payload.lng),
      type: payload.type,
      weight: finalWeight
    }
  });

  await reputationService.updateReputation(userId, payload.type, finalWeight);
  await prisma.zone.update({
    where: { id: zone.id },
    data: { demand: { increment: Math.max(1, Math.round(finalWeight)) } }
  });

  return feedback;
}

export default { submitFeedback };

================================================================================

ARQUIVO: src/modules/metrics/metrics.controller.ts

const prisma = require("../../../prisma");

async function getMetrics(req: any, res: any) {
  try {
    const userId = req.user?.id;
    if (!userId) {
      return res.status(401).json({ success: false, error: "Usuário não autenticado" });
    }

    const sessions = await prisma.strategySession.findMany({
```

```
      where: { userId },
      orderBy: { createdAt: "desc" }
    });

    const totalTime = sessions.reduce((sum: number, session: any) => sum + (session.totalTimeMin || 0), 0);
    const totalScore = sessions.reduce((sum: number, session: any) => sum + (session.totalScore || 0), 0);
    const averageScore = sessions.length ? totalScore / sessions.length : 0;
    const efficiency = totalTime ? totalScore / totalTime : 0;

    const performance = sessions.map((session: any) => ({
      id: session.id,
      status: session.status,
      totalDistance: session.totalDistance,
      totalTimeMin: session.totalTimeMin,
      totalScore: session.totalScore,
      efficiency: session.totalTimeMin ? session.totalScore / session.totalTimeMin : 0,
      startedAt: session.startedAt,
      endedAt: session.endedAt
    }));

    return res.json({
      success: true,
      data: {
        performance,
        totalTime,
        averageScore,
        efficiency
      }
    });
  } catch (err: any) {
    return res.status(500).json({ success: false, error: err.message || "Falha ao carregar métricas" });
  }
}

export default { getMetrics };

================================================================================

ARQUIVO: src/modules/strategy/strategy.controller.ts

const strategyService = require("./strategy.service");

async function startStrategy(req: any, res: any) {
  try {
    const userId = req.user?.id;
    if (!userId) {
      return res.status(401).json({ success: false, error: "Usuário não autenticado" });
    }

    const data = await strategyService.startSession(userId, {
      distance: req.body.distance,
      timeMin: req.body.timeMin,
      score: req.body.score
    });

    return res.json({ success: true, data });
  } catch (err: any) {
    return res.status(400).json({ success: false, error: err.message || "Falha ao iniciar estratégia" });
  }
}

async function updateStrategy(req: any, res: any) {
  try {
    const userId = req.user?.id;
    if (!userId) {
      return res.status(401).json({ success: false, error: "Usuário não autenticado" });
    }

    const data = await strategyService.updateSession(userId, {
      sessionId: req.body.sessionId,
      distance: req.body.distance,
      timeMin: req.body.timeMin,
      score: req.body.score
    });

    return res.json({ success: true, data });
  } catch (err: any) {
    return res.status(400).json({ success: false, error: err.message || "Falha ao atualizar estratégia" });
  }
}

async function endStrategy(req: any, res: any) {
  try {
    const userId = req.user?.id;
    if (!userId) {
      return res.status(401).json({ success: false, error: "Usuário não autenticado" });
    }

    const data = await strategyService.endSession(userId, {
      sessionId: req.body.sessionId
    });

    return res.json({ success: true, data });
  } catch (err: any) {
    return res.status(400).json({ success: false, error: err.message || "Falha ao encerrar estratégia" });
  }
}
```

```
export default { startStrategy, updateStrategy, endStrategy };

================================================================================

ARQUIVO: src/modules/strategy/strategy.service.ts

const prisma = require("../../../prisma");

async function startSession(userId: number, payload: any) {
  return prisma.strategySession.create({
    data: {
      userId,
      status: "active",
      totalDistance: payload.distance || 0,
      totalTimeMin: payload.timeMin || 0,
      totalScore: payload.score || 0
    }
  });
}

async function updateSession(userId: number, payload: any) {
  const session = await prisma.strategySession.findFirst({
    where: { id: Number(payload.sessionId), userId, status: "active" }
  });

  if (!session) {
    throw new Error("Sessão de estratégia não encontrada ou já finalizada");
  }

  return prisma.strategySession.update({
    where: { id: session.id },
    data: {
      totalDistance: session.totalDistance + Number(payload.distance || 0),
      totalTimeMin: session.totalTimeMin + Number(payload.timeMin || 0),
      totalScore: session.totalScore + Number(payload.score || 0)
    }
  });
}

async function endSession(userId: number, payload: any) {
  const session = await prisma.strategySession.findFirst({
    where: { id: Number(payload.sessionId), userId, status: "active" }
  });

  if (!session) {
    throw new Error("Sessão de estratégia não encontrada ou já encerrada");
  }

  return prisma.strategySession.update({
    where: { id: session.id },
    data: {
      status: "ended",
      endedAt: new Date()
    }
  });
}

export default { startSession, updateSession, endSession };

================================================================================

ARQUIVO: src/services/zone.service.ts

const prisma = require("../../prisma");

const SP_BOUNDARY = {
  north: -23.318,
  south: -24.050,
  west: -46.825,
  east: -46.312
};

const ZONE_NAMES = ["zona_sul", "zona_norte", "zona_leste", "zona_oeste", "centro"] as const;

type ZoneName = typeof ZONE_NAMES[number];

function validateInsideSP(lat: number, lng: number) {
  if (typeof lat !== "number" || typeof lng !== "number") return false;
  return lat <= SP_BOUNDARY.north && lat >= SP_BOUNDARY.south && lng >= SP_BOUNDARY.west && lng <= SP_BOUNDARY.east;
}

function getZoneNameFromPoint(lat: number, lng: number): ZoneName {
  if (!validateInsideSP(lat, lng)) {
    throw new Error("Ponto fora da cidade de São Paulo");
  }

  const centerLat = -23.55;
  const centerLng = -46.65;
  const latDelta = Math.abs(lat - centerLat);
  const lngDelta = Math.abs(lng - centerLng);

  if (latDelta < 0.05 && lngDelta < 0.05) {
    return "centro";
  }

  const isNorth = lat >= centerLat;
  const isEast = lng >= centerLng;
```

```
  if (isNorth && isEast) return "zona_leste";
  if (isNorth && !isEast) return "zona_norte";
  if (!isNorth && isEast) return "zona_sul";
  return "zona_oeste";
}

async function getZoneFromPoint(lat: number, lng: number) {
  const name = getZoneNameFromPoint(lat, lng);

  const zone = await prisma.zone.upsert({
    where: { name },
    create: { name, demand: 0 },
    update: {}
  });

  return zone;
}

export { validateInsideSP, getZoneFromPoint, getZoneNameFromPoint, ZONE_NAMES };

================================================================================

ARQUIVO: src/services/reputation.service.ts

const prisma = require("../../prisma");

const FEEDBACK_SCORE = {
  GOOD: 3,
  BAD: -2,
  DANGER: -4,
  TRAFFIC: 1,
  HIGH: 2
};

async function getOrCreateReputation(userId: number) {
  let reputation = await prisma.userReputation.findUnique({ where: { userId } });

  if (!reputation) {
    reputation = await prisma.userReputation.create({
      data: {
        userId,
        score: 0,
        feedbackCount: 0
      }
    });
  }

  return reputation;
}

function reputationWeight(reputation: any) {
  const base = Math.min(2, Math.max(0.5, 1 + reputation.score / 100));
  return base;
}

async function updateReputation(userId: number, type: string, weight: number) {
  const reputation = await getOrCreateReputation(userId);
  const delta = (FEEDBACK_SCORE[type] ?? 0) * (weight / 10);
  const nextScore = Math.max(-100, Math.min(100, reputation.score + delta));

  return prisma.userReputation.update({
    where: { userId },
    data: {
      score: nextScore,
      feedbackCount: reputation.feedbackCount + 1
    }
  });
}

export default { getOrCreateReputation, reputationWeight, updateReputation };

================================================================================

ARQUIVO: src/services/antifraud.service.ts

const prisma = require("../../prisma");

const blockedUsers = new Map();

async function checkFeedbackAllowed(userId: number) {
  const blockedUntil = blockedUsers.get(userId);
  if (blockedUntil && blockedUntil > new Date()) {
    return { allowed: false, reason: "Usuário temporariamente bloqueado por envio repetido" };
  }

  const windowStart = new Date(Date.now() - 10 * 60 * 1000);
  const recentCount = await prisma.hotspotFeedback.count({
    where: {
      userId,
      createdAt: { gte: windowStart }
    }
  });

  if (recentCount >= 8) {
    blockedUsers.set(userId, new Date(Date.now() + 5 * 60 * 1000));
    return { allowed: false, reason: "Excesso de feedback detectado" };
  }
```

```
  const weightFactor = recentCount >= 4 ? 0.5 : 1;
  return { allowed: true, weightFactor };
}

export default { checkFeedbackAllowed };
```

```
================================================================================
```

ARQUIVO: src/utils/cluster.util.ts

```
export interface ClusterPoint {
  lat: number;
  lng: number;
  flowId?: number;
}

export interface Cluster {
  points: ClusterPoint[];
  center: { lat: number; lng: number };
  total: number;
  radius: number;
}

function toRad(value: number) {
  return (value * Math.PI) / 180;
}

export function haversineDistance(a: ClusterPoint, b: ClusterPoint) {
  const R = 6371000;
  const dLat = toRad(b.lat - a.lat);
  const dLng = toRad(b.lng - a.lng);
  const sinLat = Math.sin(dLat / 2);
  const sinLng = Math.sin(dLng / 2);

  const h = sinLat * sinLat + Math.cos(toRad(a.lat)) * Math.cos(toRad(b.lat)) * sinLng * sinLng;
  return R * 2 * Math.atan2(Math.sqrt(h), Math.sqrt(1 - h));
}

export function groupNearbyPoints(points: ClusterPoint[], maxDistance = 250) {
  const used = new Array(points.length).fill(false);
  const clusters: Cluster[] = [];

  for (let i = 0; i < points.length; i += 1) {
    if (used[i]) continue;
    used[i] = true;

    const clusterPoints = [points[i]];
    let center = { lat: points[i].lat, lng: points[i].lng };

    for (let j = i + 1; j < points.length; j += 1) {
      if (used[j]) continue;
      if (haversineDistance(points[i], points[j]) <= maxDistance) {
        used[j] = true;
        clusterPoints.push(points[j]);
      }
    }

    const total = clusterPoints.length;
    const averageLat = clusterPoints.reduce((sum, item) => sum + item.lat, 0) / total;
    const averageLng = clusterPoints.reduce((sum, item) => sum + item.lng, 0) / total;

    center = { lat: averageLat, lng: averageLng };

    const radius = clusterPoints.reduce((max, item) => {
      const distance = haversineDistance({ lat: averageLat, lng: averageLng }, item);
      return Math.max(max, distance);
    }, 0);

    clusters.push({ points: clusterPoints, center, total, radius: Math.max(radius, 50) });
  }

  return clusters;
}

export function calculateDensityScore(cluster: Cluster) {
  const density = cluster.total / (Math.PI * Math.pow(cluster.radius / 2, 2) / 100000);
  return Math.round(Math.min(100, cluster.total * 12 + density * 6));
}
```

```
================================================================================
```

ARQUIVO: src/modules/copilot/routes.js

```
const express = require("express");
const router = express.Router();

const prisma = require("../../../prisma");
const auth = require("../../middlewares/auth");

const {
  clusterFlows,
  scoreCluster,
  predictCluster
} = require("../../services/cluster.services");

const { getHotspotPlace } = require("../../services/places.services");
const { generateInstruction } = require("../../services/decision.service");
```

```
// ================= COPILOT MULTI HOTSPOTS =================
router.get("/", auth, async (req, res) => {
  try {

    // ================= 1. FLOWS =================
    const flows = await prisma.flow.findMany({
      where: {
        createdAt: {
          gte: new Date(Date.now() - 60 * 60 * 1000)
        }
      }
    });

    const validFlows = flows.filter(f => f.latitude && f.longitude);

    if (validFlows.length === 0) {
      return res.json({ zones: [] });
    }

    // ================= 2. CLUSTERS =================
    const clusters = clusterFlows(validFlows, 200);

    // ================= 3. ENRIQUECER =================
    const enriched = clusters.map(c => {
      const metrics = scoreCluster(c);
      const prediction = predictCluster(c);

      return {
        ...c,
        ...metrics,
        ...prediction
      };
    });

    // ================= 4. AGRUPAR POR ZONA =================
    const zonesMap = {};

    enriched.forEach(c => {
      const zone = c.zone || "unknown";

      if (!zonesMap[zone]) {
        zonesMap[zone] = [];
      }

      zonesMap[zone].push(c);
    });

    // ================= 5. PROCESSAR CADA ZONA =================
    const zonesResult = [];

    for (const [zone, clusterList] of Object.entries(zonesMap)) {

      // ordenar por score
      clusterList.sort((a, b) => b.score - a.score);

      // pegar TOP 3
      const topClusters = clusterList.slice(0, 3);

      const hotspots = [];

      for (const cluster of topClusters) {

        const { lat, lng } = cluster.center;

        const place = await getHotspotPlace(lat, lng);

        const instruction = generateInstruction(cluster, place);

        hotspots.push({
          name: place.name,
          address: place.address,
          lat: place.lat,
          lng: place.lng,

          score: cluster.score,
          predicted: cluster.predicted,
          trend: cluster.trend,
          flows: cluster.total,
          recent: cluster.recent,

          instruction
        });
      }

      zonesResult.push({
```

```
================================================================================

ARQUIVO: src/modules/flow/routes.js

const router = require("express").Router();
const auth = require("../../middlewares/auth");
const controller = require("./controller");

router.post("/start", auth, controller.startFlow);
router.post("/finish", auth, controller.finishFlow);
router.get("/", auth, controller.getFlows);
```

```
module.exports = router;
```

==========

ARQUIVO: src/services/ai.service.js

```
function getUserPerformance(flows) {
  if (!flows || flows.length === 0) return [];

  const map = {};

  flows.forEach(f => {
    const key = f.zone || "unknown";

    if (!map[key]) {
      map[key] = {
        total: 0,
        totalValue: 0,
        totalTime: 0
      };
    }

    map[key].total++;
    map[key].totalValue += f.value || 0;
    map[key].totalTime += f.durationMin || 1;
  });

  return Object.entries(map).map(([zone, data]) => {
    const vpm = data.totalValue / Math.max(data.totalTime, 1);

    return {
      zone,
      valuePerMin: Number(vpm.toFixed(2)),
      total: data.total
    };
  });
}

// ================= TEMPO =================
function getTimeFactor() {
  const hour = new Date().getHours();

  if (hour >= 11 && hour <= 14) return 1.2; // almoço
  if (hour >= 18 && hour <= 22) return 1.3; // jantar
  if (hour >= 0 && hour <= 5) return 0.6;  // madrugada

  return 1;
}

// ================= IA PERSONALIZADA =================
function rankClustersForUser(clusters, userPerf) {
  const timeFactor = getTimeFactor();

  return clusters.map(c => {

    // fallback inteligente se não houver zona
    let perf = null;

    if (c.zone) {
      perf = userPerf.find(p => p.zone === c.zone);
    }

    const personalBoost = perf ? perf.valuePerMin : 0.6;

    // 🔥 pesos calibrados (mais equilibrado)
    const finalScore =
      (c.score * 0.4) +
      (c.predicted * 0.4) +
      (personalBoost * 15 * 0.2);

    return {
      ...c,
      personalScore: Math.round(finalScore * timeFactor)
    };
  });
}

module.exports = {
  getUserPerformance,
  rankClustersForUser
};
```

==========

FIM DO DOCUMENTO